COMP90041
Programming and Software Development

Week 1

Amani Abusafia

Acknowledgement of Country

The University of Melbourne acknowledges the Traditional Owners of the unceded land on which we work, learn and live: the Wurundjeri Woi-wurrung and Bunurong peoples (Burnley, Fishermans Bend, Parkville, Southbank and Werribee campuses), the Yorta Yorta Nation (Dookie and Shepparton campuses), and the Dja Dja Wurrung people (Creswick campus).

The University also acknowledges and is grateful to the Traditional Owners, Elders and Knowledge Holders of all Indigenous nations and clans who have been instrumental in our reconciliation journey.

We recognise the unique place held by Aboriginal and Torres Strait Islander peoples as the original owners and custodians of the lands and waterways across the Australian continent, with histories of continuous connection dating back more than 60,000 years. We also acknowledge their enduring cultural practices of caring for Country.

We pay respect to Elders past, present and future, and acknowledge the importance of Indigenous knowledge in the Academy. As a community of researchers, teachers, professional staff and students we are privileged to work and learn every day with Indigenous colleagues and partners.

THE UNIVERSITY OF MELBOURNE

Topics:

- Introduction to the Subject
- Getting Started - Java

Agenda

- Introduction to the Subject
- Teaching Staff
- Assessments
- Academic Misconduct
- How to navigate Ed
- Subject Overview
- What's New?
- Getting Started With Java
- Setting up your Java Environment
- Data Types and Console I/O
- Operators

About This Subject

- **What is this subject?**
  - An introduction to programming and software development using Java
- **What will you learn?**
  - Programming in Java, Object-Oriented Programming, Reading and analysing requirements, testing, and software design
- **Why does it matter?**
  - Java and OOP skills are industry-standard; used across software, finance, healthcare, and more

Lecturers

Amani Abusafia

Lecturer & Subject Coordinator
BSc Comp Sci (2009), MS Comp Sci (2013), PhD in Eng (2024)

Worked in Academia for 15+ years

C++, Java, Python, SQL,...

Danny Xu

Lecturer and Research Fellow
BSc Eng (2012), MS Eng (2015), PhD in Comp Sci (2022)

Research in computer vision and deep learning

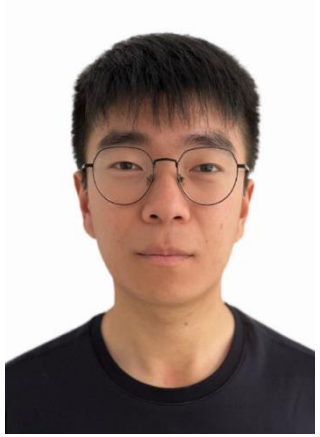
C++/CUDA, PyTorch, MATLAB, …

Teaching Staff

Trina Dey
(Academic Specialist)

Sandhuni
(head tutor)

Marjan



Haoyu Gao

Nadeeshan

Xunye Tian

Subject Logistics

- **Handbook entry:**
  - https://handbook.unimelb.edu.au/subjects/comp90041
- **Weekly commitments:**
  - Lectures: 2 hours (Fri, 11-1pm, JH Michell Theatre)
  - Tutorials: 2 hours (1 hour tutorial, 1-hour self-practice)
- **Subject Content**
  - **Canvas:** modules + lecture capture
  - **EdStem:** lessons + assignments + discussions
    - All discussions are on EdStem (not Canvas)
  - **Textbook:**
    - Walter Savitch, Absolute Java, 6th Edition, Addison Wesley.
    - SCHILDT, H. Java: The Complete Reference, 12th Edition: McGraw-Hill, 2022

Getting to know you!

THE UNIVERSITY OF MELBOURNE

PollEv.com/abusafia

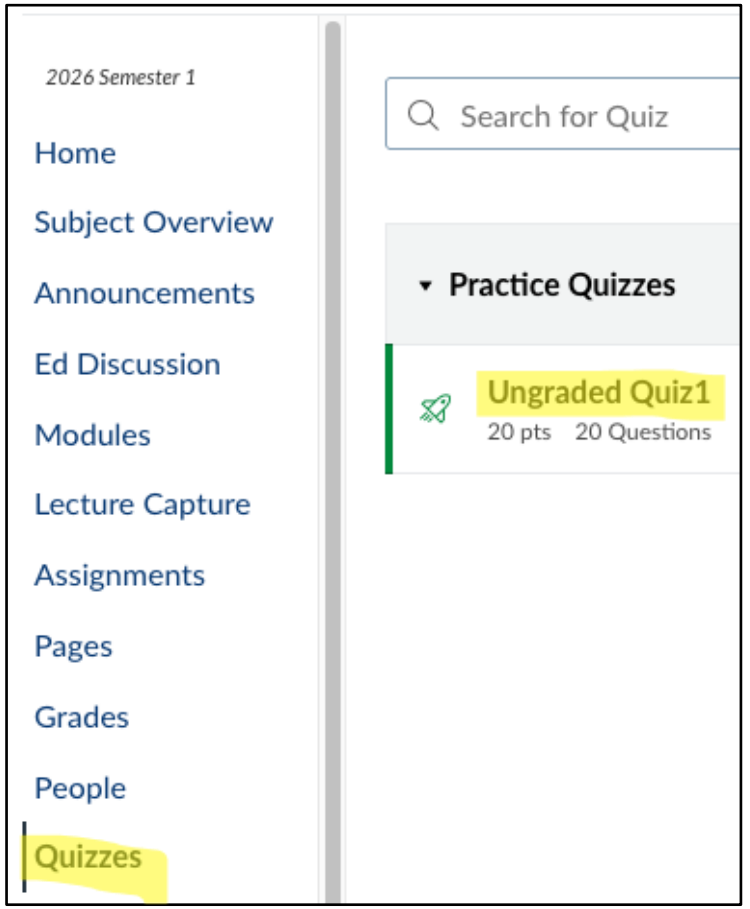
Ungraded Quiz – What It Means

Purpose

- The quiz is ungraded. The goal is self-awareness, not evaluation.
- It helps you understand your current level of programming logic.
- Use it to identify concepts you may want to review as we begin learning Java.

How to interpret it

- If you found many questions easy, you already have a good foundation in programming concepts. (even if repeated the quiz)
- If some questions were difficult, that is normal and simply shows where you may need more practice.
- If you have no idea what is going on, you need to spend extra time reviewing the fundamentals.



Assessments

Assessments in 2026, semester 1, consist of:

Type	Week	Duration	Start Date	Due Date	%age
Practice Mid Sem Quiz	Week 4	30 min	27 Mar 26 17:00 PM	-	0
Mid Sem Quiz	Week 6	30 min	15 Apr 26 16:30 PM	-	10
Assignment 1	Week 5-7 (including Mid Sem Break)	3 weeks	02 Apr 26 09:00 AM	22 Apr 26 11:59 PM	20
Assignment 2	Week 9-12	3 weeks	08 May 26 09:00 AM	29 May 26 11:59 PM	30
Final Exam	During Exam Weeks	2 hour	TBD	TBD	40

The quizzes and exams will run online, using the ***Lockdown Browser*** to ensure integrity.

Hurdle Requirements

Hurdle	Requirement
Exams	At least 20 marks combined (Mid-Sem + Final)
Assignments	At least 20 marks combined (A1 + A2)
Overall	At least 50 marks of the total subject

See handbook for more information:
https://handbook.unimelb.edu.au/subjects/comp90041/assessment

Extensions & Special Consideration — Assignments

Type	Duration	What to Do	Approver
Short Extension	Up to 3 days	Complete **FEIT Short Extension** form	Subject Coordinator
Long Extension	4–10 days	Submit **Special Consideration** form with supporting evidence	Special Consideration
AAP	Pre-approved days	Notify Subject Coordinator	Student Equity

When to apply? Before Assessment Due date

See details in this module - here

Extensions & Special Consideration — Exams

Assessment	What to Do	Approver	Managing
Mid Sem Quiz	Submit **Special Consideration** form with supporting evidence	Special Consideration	Subject Coordinator
Exam	Submit **Special Consideration** form with supporting evidence	Special Consideration	Centralised Exam team
AAP	Notify Subject Coordinator	Student Equity	Depends on assessment

When to apply? No later than 4 business days after the exam

See details in this module - here

Academic Misconduct

- **Plagiarism**
  - Modifying else's code to make your own
- **Cheating**
  - Copying else's code
  - Copying from internet
  - Outsourcing your project
- **Collusion**
  - Two or more people in the class working together
  - Getting help from outside the university
- **Recycling**
  - Using your own work from elsewhere without notifying
- **Violation of IP**
  - Outsourcing your project
  - Sharing your code/assignment on Github or public platforms

Academic Misconduct

Read Later

Teaching Team detects Academic Misconduct

Notify School Liaison for Misconduct

Once approved, notify Misconduct Committee

Committee meets Student

Outcome decided/ notify Teaching Team

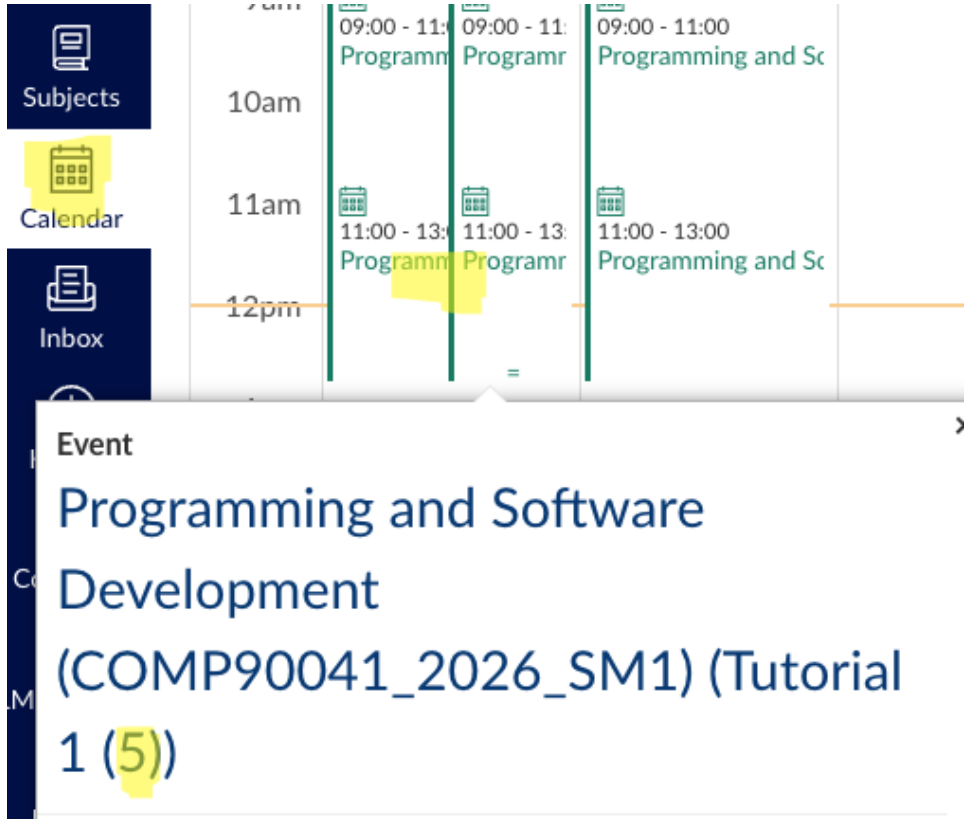
Tentative Subject Schedule

Week 1	Week 2	Week 3	Week 4	Week 5	Mid Sem Break	Week 6
▪ Subject Overview ▪ I/O ▪ Data Types ▪ Operators	▪ Control Flow Labor Day holiday on Monday	▪ Classes	▪ Classes II	▪ Arrays Good Friday on Friday	▪ **Non-Teaching Period**	▪ OOPs
Week 7	**Week 8**	**Week 9**	**Week 10**	**Week 11**	**Week 12**	**SWOT VAC**
▪ OOP continued	▪ Java Interfaces	▪ Exception Handling	▪ File Handling	▪ Generics ▪ Collections	▪ Advanced Topics (Non-Examinable) ▪ Revision	**Non-Teaching Period**

Tutorial Rescheduling – Public Holiday (Monday)

Week 2 only for tutorials on Monday

Tutorial	Old Time	New Time	Location
T01	Monday 9:00 AM	Wednesday 10:00 AM	PAR-192-L2-211 – Flexible Learning Space (35)
T02	Monday 9:00 AM	Wednesday 9:00 AM	PAR-200-L2-208 – VIEPS Intensive Room (36)
T04	Monday 11:00 AM	Tuesday 9:00 AM	PAR-104-G-G01 – Collaborative Learning Space (30)
T05	Monday 11:00 AM	Thursday 10:00 AM	PAR-192-L2-211 – Flexible Learning Space (35)
T08	Monday 6:00 PM	Tuesday 4:00 PM	PAR-149-L1-156 – Collaborative Learning Space (30)
T09	Monday 6:00 PM	Thursday 3:00 PM	PAR-104-G-G03 – Collaborative Learning Space (30)



Getting the Most Out of This Course

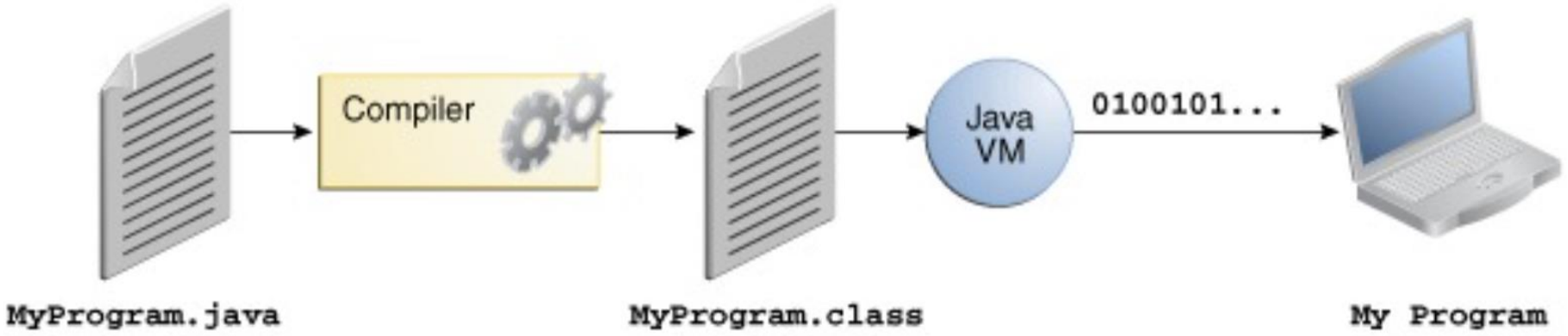
- **Be Proactive**
  - Pay attention to the learning outcomes in each lecture
  - Regularly check your progress against course objectives
- **Manage Your Time**
  - Plan a head for assignments and assessments
  - Break tasks into smaller steps to avoid last-minute stress
- **Engage with Others**
  - Collaborate with classmates—discuss ideas and challenges
- **Ask for Help Early**
  - If you're struggling, don't wait—reach out for support
  - Use available resources like Ed and consultations
- **Embrace the new concepts and enjoy the learning process!**

Getting Started – Overview

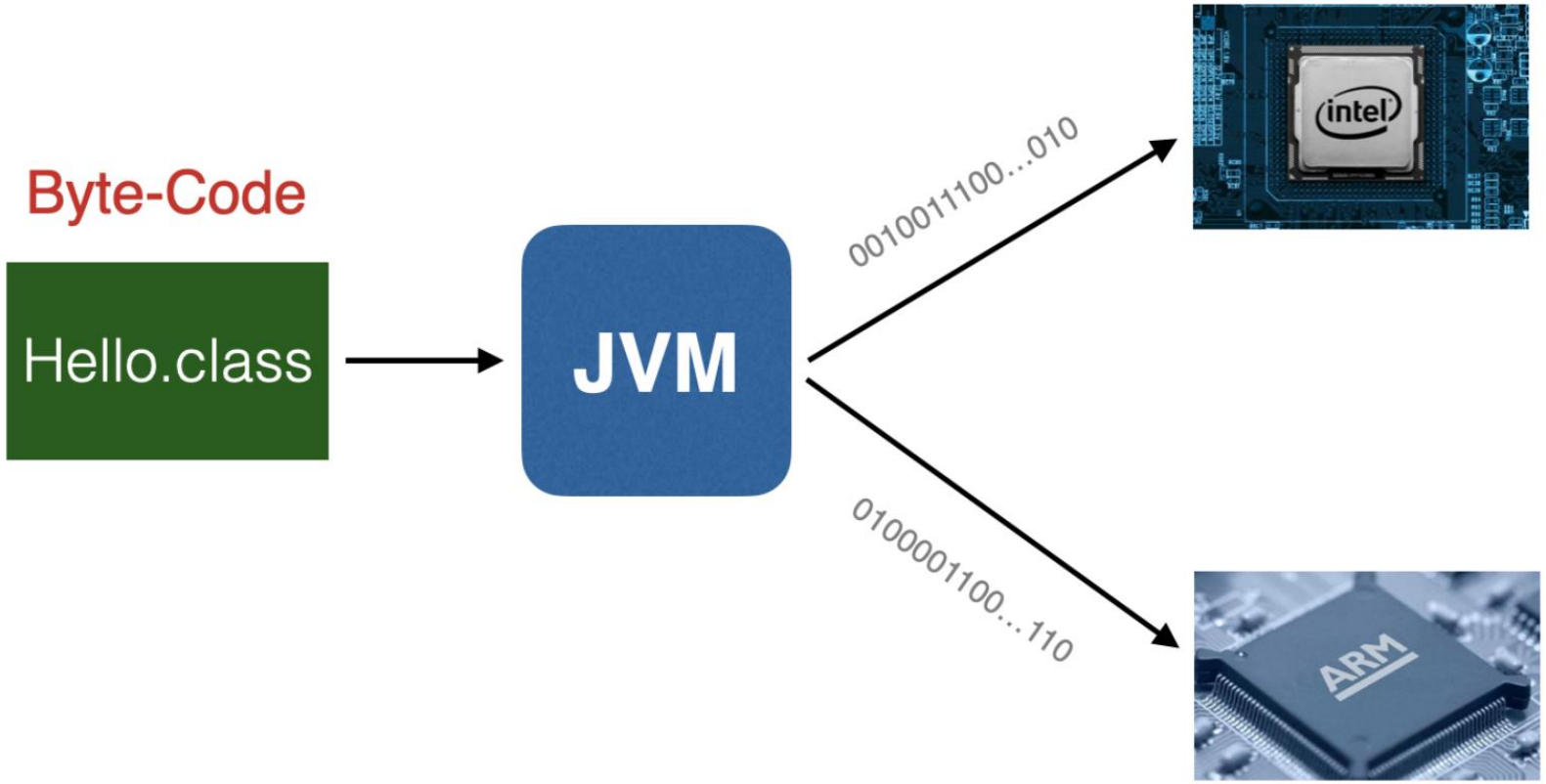
In this part of the lecture, you will learn about:

- **First Program**
- **Compilers and Interpreters**
- **Data Types**
- **I/O**
- **Operations**

Compilers & Interpreters



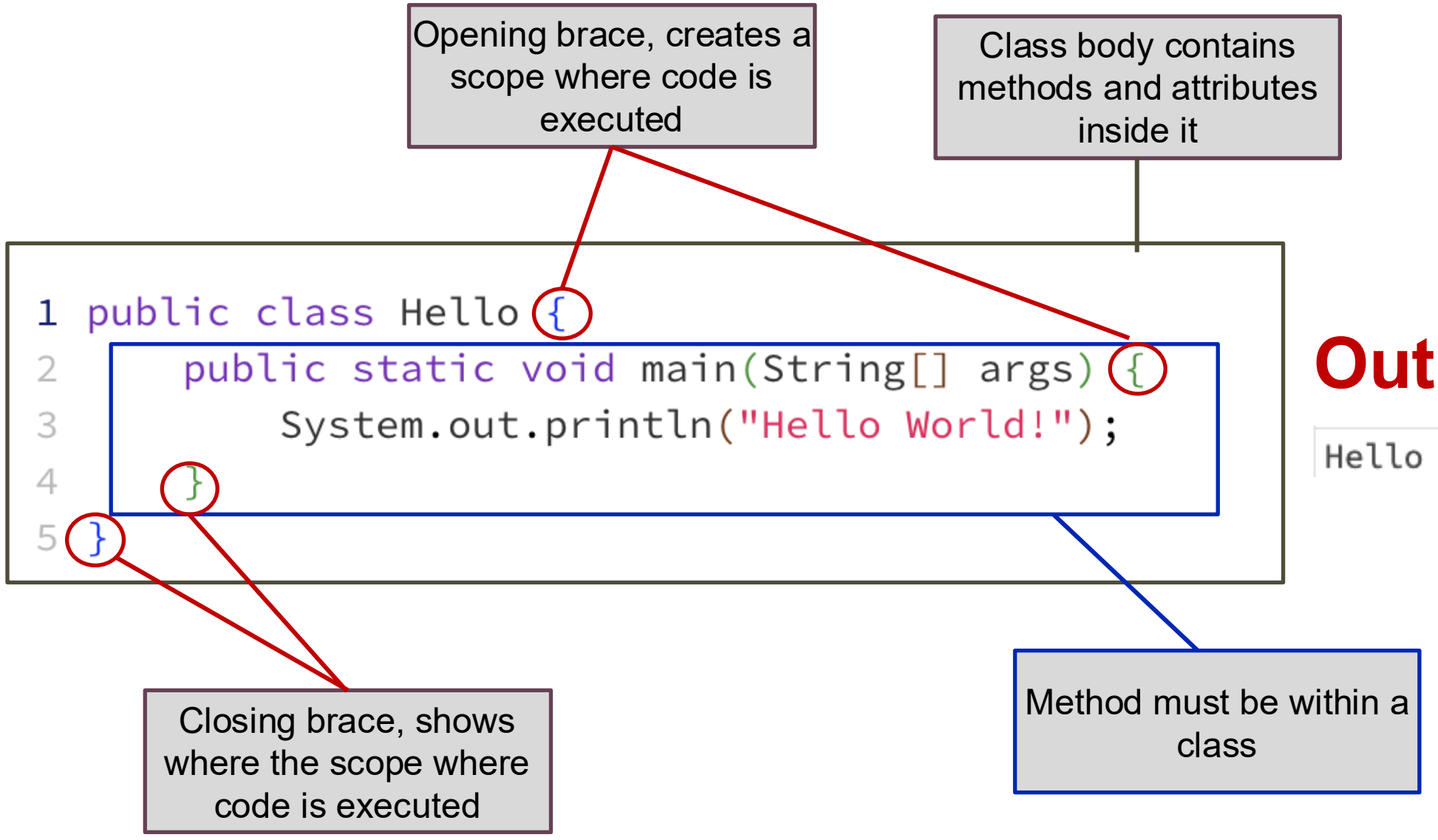
Compilers & Interpreters



Compilers & Interpreters

- Java compiler – javac
- JVM – Java Virtual Machine – helps execute bytecode. Two ways
  - Interpreter – interprets the bytecode to machine executable code
  - JIT – Just in Time Compiler – compiles the bytecode into executeable code on need basis
- JRE – Java Runtime Environment – System libraries like you used to print Input/Output
- JDK – Java Development Kit - Helps you to write or develop the programs. Provides series of system libraries.

First Program – Hello World!



Output

```
Hello World!
```

First Program – Hello World!

Public – accessible to all
Private – hidden from some

Everything goes in a class in Java

Every class has a name

Message we are trying to print

```
1 public class Hello {
2     public static void main(String[] args) {
3         System.out.println("Hello World!");
4     }
5 }
```

We will discuss static more in Week 4

main is the starting point for any program. Java compilers looks for a method named main

System – java library
Out – refers to standard output
Println – prints with newline at the end

General Code Structure


```
class ClassName {
        public static void main (String [] arguments) {
         → Your code goes here ←
        }
}
```

Data Types

- In general, a computer program consist of two parts: **code and data.**
- **Code** is the text of the program that determines what operations the program performs.
- **Data** is what the code operates on.
  - Each datum (singular of data) has a **type**.
- Java distinguishes between three groups of data types: **primitive, class, and array**.

Data Types - Primitive

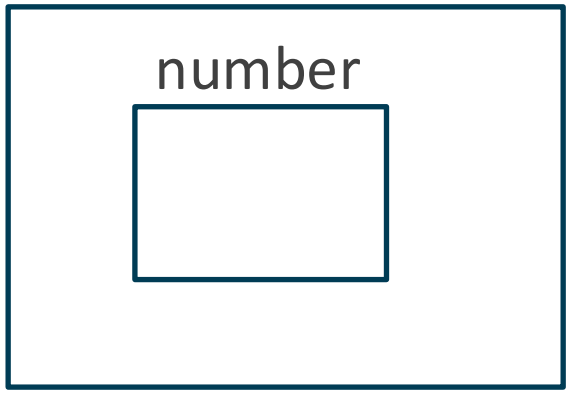
Type	Size (Bytes)	Contains	Values (Range)	Example	Default values (for fields)
boolean	not precisely definited, typically 1 bit but size is JVM dependent	boolean values true or false	-	boolean isStudent = true;	false
char	2 (16 bits)	unicode characters	\u0000' (or 0) to '\uffff' (or 65,535 inclusive)	char c = 'c';	\u0000'
byte	1 (8 bits)	signed integer	-128 to 127	bytes b = 100;	0
short	2 (16 bits)	signed integer	-32,768 to 32,767	short s = 1000;	0
int	4 (32 bits)	signed integer	-2,147,483,648 to 2,147,483,647	int i = 1000000;	0
long	8 (64 bits)	signed integer	-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807	long l = 100000000L;	0
float	4 (32 bits)	IEEE 754 floating point	±3.40282347E+38F (6-7 significant decimal digits)	float f = 1.45f;	0.0f
double	8 (64 bits)	IEEE 754 floating point	±1.79769313486231570E+308 (15 significant decimal digits)	double d = 1.457891d;	0.0d

Variables

- Variables are named locations that store data
- FORMAT:

```
type variableName;
int number;
```

- Variables can have different values at different times.
- In Java, variable names begin with a letter, followed by letters, digits, and underscores (_).
- Best practice: make them descriptive, but not too long (clear abbreviations are okay)
- Each variable must be declared before being used, specifying its type.



Memory

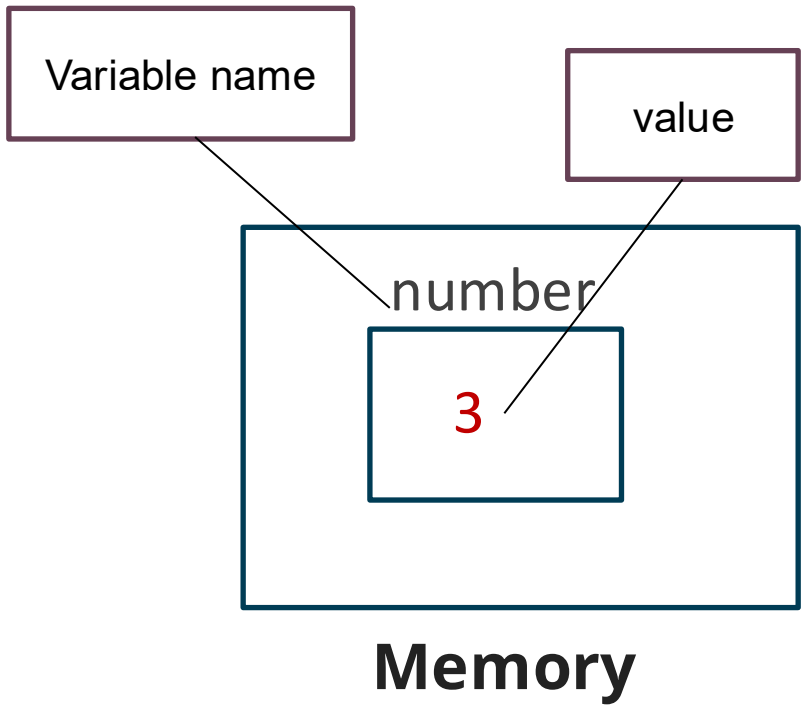
Variables

- **Variables must be assigned a value before being used.**
- **This can be done in two ways:**

```
int number;
number = 3;
```

- **OR Declaration and assignment can also happen in a single statement:**

```
int number = 3;
```



I/O: Standard Output

```
System.out.print("Prints some characters");
System.out.println("Prints a newline at the end.");
System.out.println("A new line");
```

The Output will be:

```
Prints some charactersPrints a newline at the end.
A new line
```

I/O: Formatted Output


```
double average = 5.0;
System.out.printf("Average: %5.2f", average);
```

The Output will be:

```
Average: 5.00
```

I/O: Formatted Output

```
String s = "string";
System.out.printf("\"%s\" has %d characters %n", s, 6);
```

The Output will be:

```
"string" has 6 characters
```

I/O: Standard Input

Include the scanner library using import statement

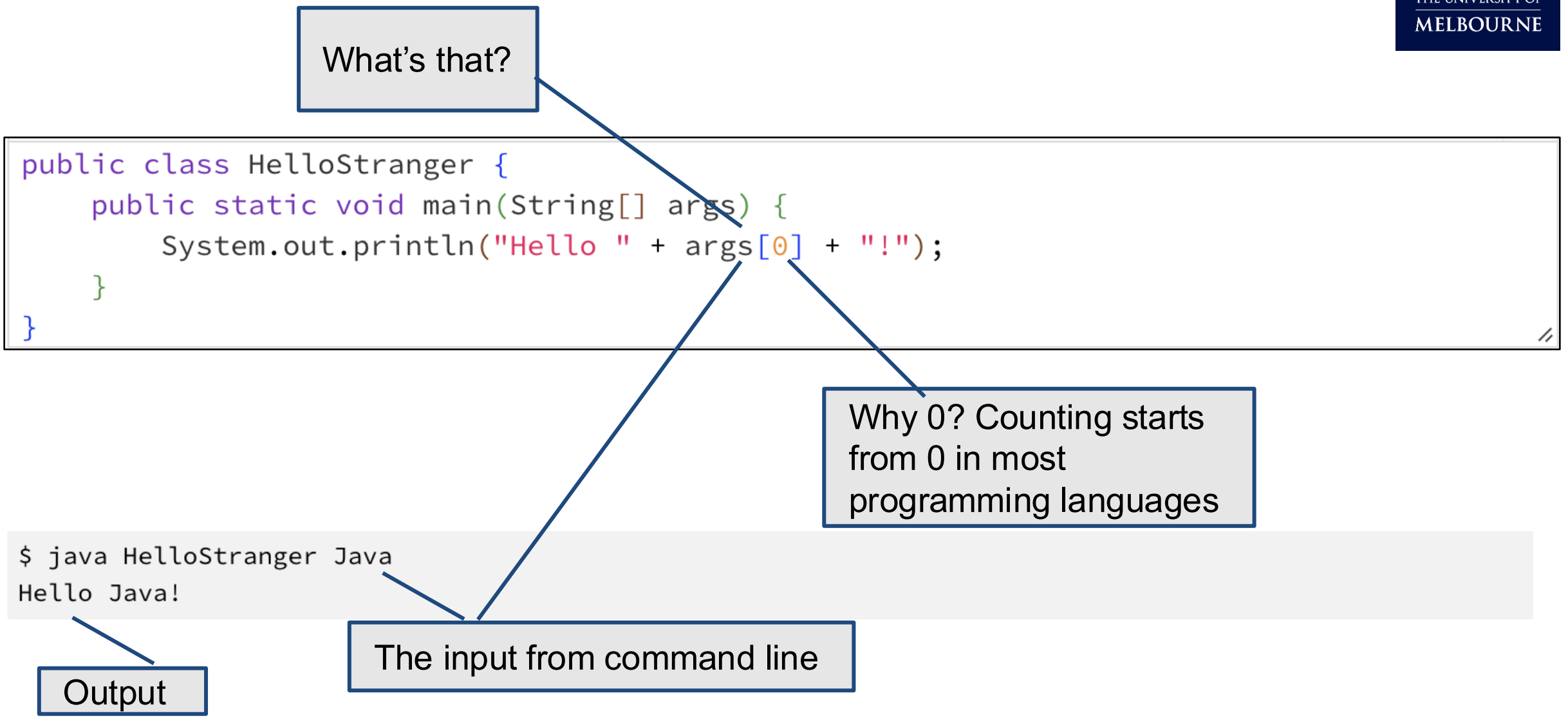
Write the first line defining the scanner named as keyboard.

Scanner scans System.in

Read the line from the keyboard

```
1  import java.util.Scanner;
2
3  public class ScannerPlay {
4      public static void main(String[] args) {
5          Scanner keyboard = new Scanner(System.in);
6          System.out.print("Please enter your name: ");
7          String name = keyboard.nextLine();
8          System.out.println("Hello " + name + "!");
9      }
10 }
```

I/O: Command Line Input

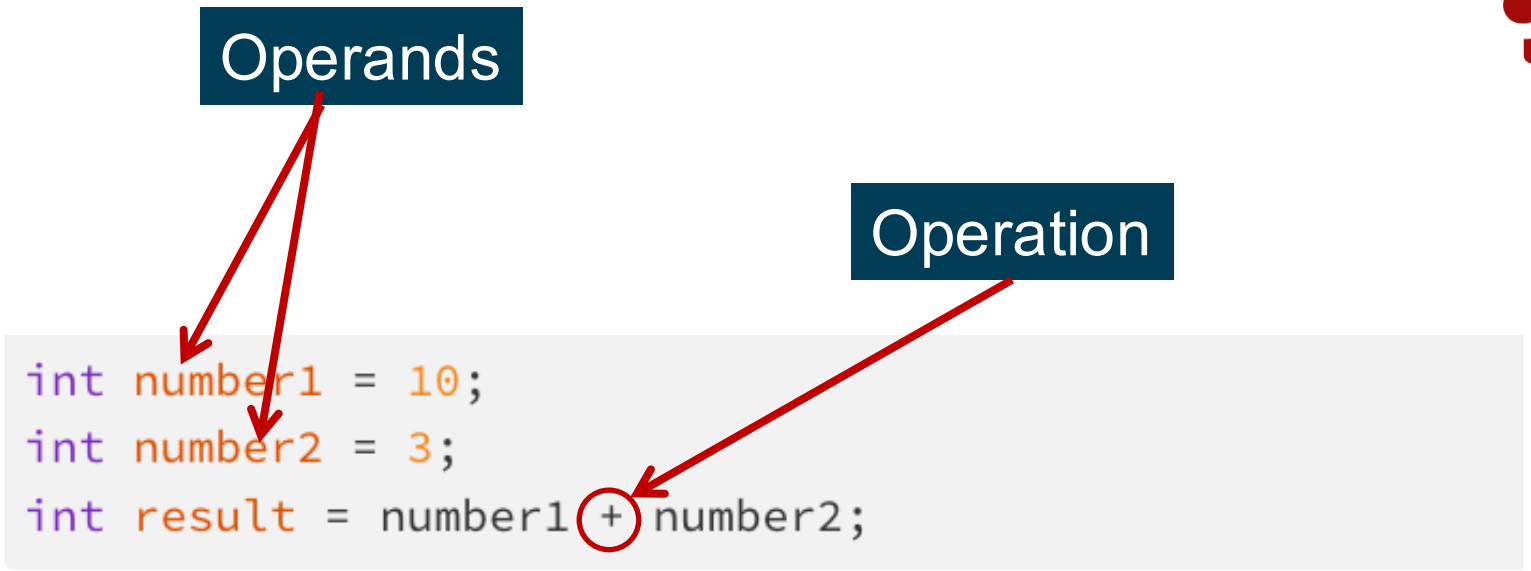


Number Types

- Each type has certain operations that apply to it.
- Common ***operations (also called operators)*** for primitive number types are:
  - Addition (+) and Subtraction (-)
  - Multiplication (*) and Division (/) as well as Modulo (%), i.e., the remainder

- The numbers we apply the operation to are called the ***operands***.
- The **data type of the result** is the same as the type of the operands.

Number Types



Operations are used to construct ***expressions***, which have values that can be assigned or used as operands:

```
int answer=(2+4)*7;
```

Integer vs Float Division

Integer Division

```
System.out.println(5/2);
```

Float Division

```
System.out.println(5.0/2);
```

The Output will be:

```
2
```

```
2.5
```

Comparison Operations

- The following **comparison operations** also work for number types:

```
<  : less than
<= : less than or equal to
>  : greater than
>= : greater than or equal to
== : equal to
!= : not equal to
```

- Comparisons always **return** a **boolean (true / false)** value

```
System.out.println(5!=4);
```

```
Output is True
```

Operations for Booleans

- You can construct **logic statement** in Java using:

```
&& (AND) : is true if both operands are true
|| (OR) : is true if either operand is true
```

- Both of these are so-called **'short-circuit'** operations, *i.e.*, the second **operand** is only evaluated if necessary.
  - If the first argument of && is false, or the first argument of || is true, then the second one is not necessary because it cannot change the value of the expression.
- Then, there is also **negation**:

```
! (NOT) : is true if its operand is false
```

Increments and Decrements

- Increments and decrements are a **special expressions**
- There are **two** types of increments and decrements:

pre

- The **pre-increment** increments a number by **1 before** returning the value.

++y

- The **pre-decrement** - decrements a number by **1 before** returning it:

-- y

post

- The **post-increment** increments a number by **1 after** returning the value.

y++

- The **post-decrement** - decrements a number by **1 after** returning it:

y--

Increments and Decrements

```
int x = 5;
```

pre

```
System.out.println(++x);
```

```
Output is 6
```

post

```
System.out.println(x++);
```

```
Output is 5
```

- Pre/post increment/decrements can also be used as statements rather than expressions:

```
++x;        OR        x++;
```

Type Conversions

- **Primitive** operations generally work on operands of the **same** type.
- Java can, however, convert types if the operands have different types.
  - A **widening conversion** converts a number to a wider type so the value can always be converted **automatically**
  - A **narrow conversion** converts a larger data type to a smaller one **explicitly**.

Type Conversions

- A **widening conversion** converts a number to a wider type so the value can always be converted successfully:
- Java converts **automatically** between the following:

`byte` ➔ `short` ➔ `int` ➔ `long` ➔ `float` ➔ `double`

```
int x = 5;
long y = 10;
float z = 20;
System.out.println(x+y);
System.out.println(x+z);
```

The Output will be:

```
15
25.0
```

Char Type Conversions

- The ***char*** type is a special case.
- While it technically is **not an *int*** it is an **integral type**, *i.e.,* it is considered to be a whole number that can be converted to and from other integral types:

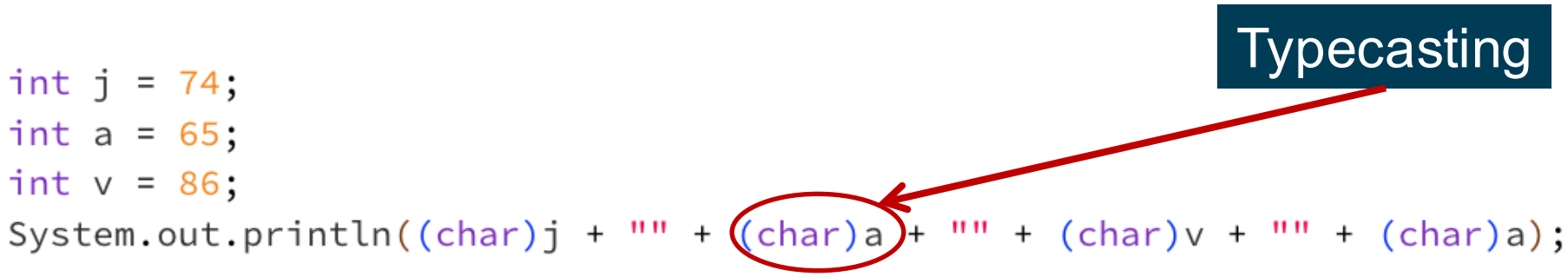
```
char c = 'A';
long y = 10;
System.out.println(c+y);
```

The Output will be: 75

Character	ASCII Code
A	65
B	66
C	67
D	68
E	69
F	70
G	71
H	72
I	73
J	74
K	75
L	76
M	77
N	78
O	79
P	80
Q	81
R	82
S	83
T	84
U	85
V	86
W	87
X	88
Y	89
Z	90

Char Type Conversions

- A *char* converted to an ***int*** represents the **corresponding ASCII code** (or Unicode code point) of the character.
- **Typecasting** can be used to convert an *int* to a *char* type:



The Output will be: `JAVA`

- Such casting is a **narrowing conversion**.

Character	ASCII Code
A	65
B	66
C	67
D	68
E	69
F	70
G	71
H	72
I	73
J	74
K	75
L	76
M	77
N	78
O	79
P	80
Q	81
R	82
S	83
T	84
U	85
V	86
W	87
X	88
Y	89
Z	90

Char Type Conversions

- A **narrow conversion** converts a larger data type to a smaller one **explicitly**.
- It needs to be **specified by writing the name of the type** to convert to in parentheses before the value that is to be converted.
- A cast can also be used to explicitly ask for a widening conversion

```
int sum = 10;
int count = 2;
double average = (double)sum / count;
System.out.println(average);
```

Typecasting

The Output will be:

```
5.0
```

Precedence and Associativity

- **Operator precedence** determines the order in which the operators in an expression are evaluated.

Precedence of two operators, say $\odot$ and $\oplus$, determines whether $a \odot b \oplus c$ is read as:

$(a \odot b) \oplus c$ ($\odot$ has higher precedence) or

$a \odot (b \oplus c)$ ($\odot$ has lower precedence)

- For example, 2+3*4=14 as * has higher precedence.

Precedence and Associativity

Associativity determines whether $a \odot b \odot c$ is read as:

$$(a \odot b) \odot c \text{ (left associativy) or}$$

$$a \odot (b \odot c) \text{ (right associativity)}$$

- For example, 3-2-1=0 (associates left).
- *If* it were associated right (which it doesn't!), then 3-2-1 would be 3-(2-1) = 3-1 = 2.

Precedence and Associativity

The following table lists the precedence of operators in Java. The **higher** it appears in the table, the **higher** its precedence:

Operators	Precendence	Associativity
Postfix	++ - -	Left to right
Unary	+ - ! ~ ++ - -	Right to left
Multiplicative	* / %	Left to right
Additive	+ -	Left to right
Shift	<< >>	Left to right
Relational	< <= > >=	Left to right
Equality	== !=	Left to right
Bitwise AND	&	Left to right
Bitwise XOR	^	Left to right
Bitwise OR	|	Left to right
Logical AND	&&	Left to right
Logical OR	||	Left to right
Conditional	?:	Right to left
Assignment	= += -= *= /= %= >>= <<= &= ^= |=	Right to left

Quick Practice

Quick Practice


```
int x = 10, y = 5, z = 2;
```

Example 1:

```
        int result1 = (x + y) * z;
What is the value of result1 ?
```

- Evaluates to 30
- **Precedence:** Parenthesis () overrides default precedence

Example 2:

```
        int result2 = x++ + x + y;
What is the value of result2 ?
```

- Evaluates to 26
- **Precedence:** Postfix then **Associativity:** Arithmetic operators associate left-to-right

```
int result2 = ++x + x + y;
```

The String Type

- *String* is a ***class (reference) type***, not a primitive type, so strings are objects.
- Strings are widely used in Java Programming and they are sequence of characters.
- A *string* constant is specified by enclosing it in double-quotes (""):

```
String greetings = "Hello World!";
```

- Using a backslash (\) allows you to include double-quotes and other special characters (including \ itself) in a string:

```
System.out.println("He said a \"backslash (\\) is special!\"");
System.out.println("Windows file names become C:\\users\\fred");
```

The Output will be:

```
He said a "backslash (\) is special!"
Windows file names become C:\users\fred
```

The String Type

- Certain letters **after a backslash** are treated specially:
- For example, **\n** for a new line and \t for a **tab character**.

```
System.out.println("I\nlike\nme\nsome\nnew\nlines!");
```

The Output will be:

```
I
like
me
some
new
lines!
```

String Operations

- Two strings can be **appended** using **+**
- This operation also called **concatenation**.
- If either operand is a **string**, the **+** operation will **convert** the other operand into a string.

Example 1:

```
System.out.println("Hello " + "String!");
```

The Output will be:

```
Hello String!
```

Example 2:

```
int x = 1;
System.out.println("x = " + x);
```

```
x = 1
```

String Operations

- The **String class** comes with a whole range of useful **operations**, which you can look up in Java Doc.

```
String s = "A piece of string walks into a bar...";

// returns length of the string
System.out.println("String length: " + s.length());

// returns ALL UPPER CASE version of the string
System.out.println("All upper case: " + s.toUpperCase());

// s.substring(i, j) returns the substring of s from character i
// through j-1, counting the first char at 0
System.out.println("A substring: " + s.substring(0, 17));
```

The Output will be:

```
String length: 37
All upper case: A PIECE OF STRING WALKS INTO A BAR...
A substring: A piece of string
```

THE UNIVERSITY OF
MELBOURNE

Quick Practice

Quick Practice

What is the output of the following program if the input was:

```
java Quiz 10 4
```

Program:

```
public class Quiz {
        public static void main(String[] args) {
                System.out.println(args[0] + args[1]);
        }
}
```

The Output will be: `104`

Additional Reading Resources

- WALTER, S. Absolute Java, Global Edition. [Harlow]: Pearson, 2016. (Chapter 1, 2 and 3)
- SCHILDT, H. Java: The Complete Reference, 12th Edition: McGraw-Hill, 2022 (Chapter 4)
- Language Basics (accessible on 14-02-2024) Oracle's Java Documentation. Available at: https://docs.oracle.com/javase/tutorial/java/nutsandbolts/index.html

THE UNIVERSITY OF MELBOURNE

See you next time!

The University of Melbourne (Australian University) PRV12150/CRICOS 00116K